

Results of Programming Assignment 3

-Ananya Asthana

Introduction

For this assignment, I have implemented 3 models:

1. Model 1: Feed-forward network using ReLu activation Function
2. Model 2: Feed-forward network using Tanh activation Function
3. Model 3: CNN network

I will report details for every model in the following sections

Other information:

1. Accuracy changes:

I started with a batch size of 64. I was getting low accuracies in the range of 20s for feed-forward and 30% for CNN. Then I reduced batch size to 34 and added another hidden layer to my CNN model and my accuracy went up to late 30s for feed-forward and 50.6% for CNN. Then I settled on a batch size of 16 and got 45+ accuracies for feed-forward and 59.34% for CNN. After this I moved to Adam optimizer instead of SGD and also added dropout and BatchNorm. These changes pushed my accuracy to 50s for feed-forward and 78.16% for CNN. But when I moved to computing training loss in evaluation mode instead of using the running loss calculated inside the training loop, my accuracy for feed-forward Tanh decreased to 46.46%. I did this because our assignment needed us to compute the loss over the entire training dataset after each epoch, which wasn't happening when I calculated loss inside the training loop, as that was based on mini-batches during training mode.

Model 1: Feed-forward neural network using ReLu activation Function

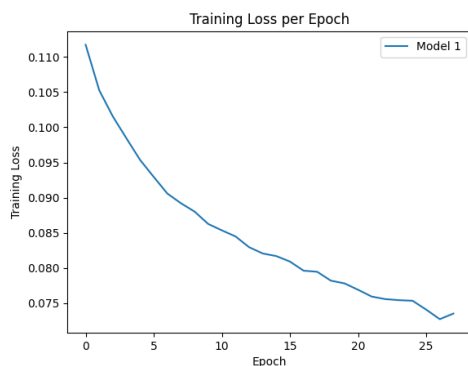
1. **Description of the network architecture:**

This is a fully-connected feedforward neural network consisting of 3 hidden layers. It uses a ReLU activation function after every linear full-connected layer. It also uses a dropout function to randomly disable 30% of the neurons during training in order to prevent overfitting and also to prevent reliance on specific neurons. The first layer has an input shape of 3x32x32 image which has been flattened, and has output features or neurons equal to 512. There is a gradual reduction in neuron counts between layers, going from 512 to 256 to 128. Finally, the output layer contains 10 neurons, corresponding to the 10 CIFAR-10 classes.

2. **Final Accuracy:**

52.68%

3. **Graph of the training loss after each epoch of training:**



4. Correctly classified image:

Correct Prediction
True: truck, Pred: truck



Actual label: truck
Predicted label: truck

5. Incorrectly classified image:

Incorrect Prediction
True: ship, Pred: cat



Actual label: ship
Predicted label: cat

Model 2: Feed-forward neural network using Tanh activation Function

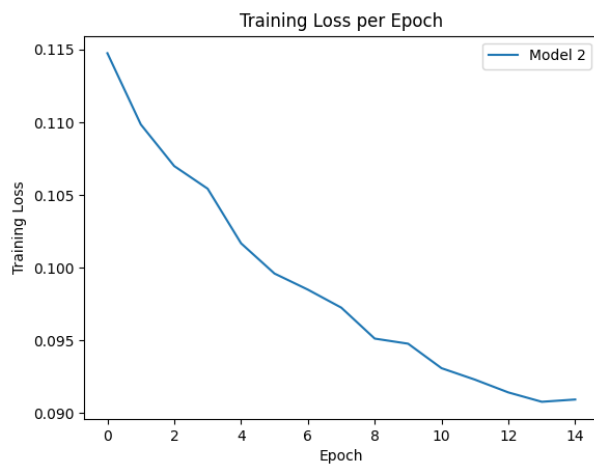
1. Description of the network architecture:

This is a fully-connected feedforward neural network consisting of 4 hidden layers. It uses a Tanh activation function after every linear full-connected layer. It also uses a dropout function to randomly disable 30% of the neurons during training in order to prevent overfitting and also to prevent reliance on specific neurons. The first layer has an input shape of 3x32x32 image which has been flattened and has output features or neurons equal to 256. There is a gradual reduction in neuron counts between layers, going from 256 to 128 to 64 to 32. Finally, the output layer contains 10 neurons, corresponding to the 10 CIFAR-10 classes.

2. Final Accuracy:

46.46%

3. Graph of the training loss after each epoch of training:



4. Correctly classified image:

Correct Prediction
True: cat, Pred: cat



Actual label: cat
Predicted label: cat

5. Incorrectly classified image:

Incorrect Prediction
True: deer, Pred: bird



Actual label: deer
Predicted label: bird

Model 3: Convolutional neural network

1. Description of the network architecture:

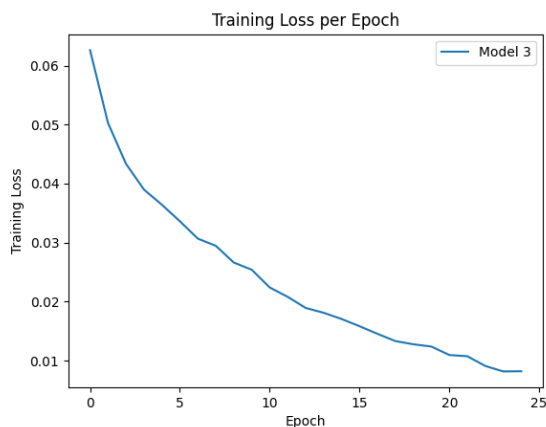
This is a convolutional neural network consisting of 4 convolutional layers, 2 MaxPooling layers, 2 fully-connected layers with dropout. It uses a ReLU activation function after every convolutional layer and batch normalization. The first convolutional layer processes a 3-channel input image, where the 3 channels are for the RGB colors, using 32 filters. After we go a few layers deep, the filters increase to 64 to capture more complex patterns. We use batch normalization to stabilize training and speed up convergence. MaxPooling layers are added after every two convolutional layers and they help the model generalize better by focusing on higher-level features. Once all the convolutional layers have been passed, the output from them is flattened into a 1D vector and fed as input to the fully-connected layers.

The first layer of the fully-connected layers has an input shape of 64x8x8 image which has been flattened, and has output features or neurons equal to 256. There is a reduction in neuron counts between layers, going from 256 to 10, where 10 corresponds to the CIFAR-10 classes. It uses a ReLU activation function and it also uses a dropout function to randomly disable 50% of the neurons during training in order to prevent overfitting and also to prevent reliance on specific neurons.

2. Final Accuracy:

79.21%

3. Graph of the training loss after each epoch of training:



4. Correctly classified image:

Correct Prediction
True: automobile, Pred: automobile



Actual label: automobile

Predicted label: automobile

5. Incorrectly classified image:

Incorrect Prediction
True: bird, Pred: cat



Actual label: bird

Predicted label: cat

Comparison of the 3 networks

Out of our 3 neural networks, Model 3, or the Convolutional Neural Network, or CNN, performed the best, with a final accuracy of 79.21%. This is because CNNs are specifically designed for image related datasets, in addition to some of the architecture decisions we made. CNNs use convolutional filters, which we gradually increased to capture more complex patterns. Convolutional filters extract local features like edges, textures and shapes. We used batch normalization to stabilize and speed up training, and dropout in the fully connected layers to prevent overfitting. We also used MaxPooling which reduces the spatial dimensions and helps the model focus on the most prominent features, ignoring noise. Also, it used ReLu as the activation function. ReLU is known for its efficiency and ability to mitigate vanishing gradient issues. For all these reasons I think this network performed the best.

Model 2, which was our feed-forward network using Tanh activation function performed the worst with an accuracy of 46.46%. Even though it had an additional layer compared to Model 1, it used Tanh which is more prone to vanishing gradients, and therefore did not perform as well as Model 1 which used ReLU and had an accuracy of 52.68%, and definitely not as good as Model 3 which was our best-performing CNN. It also started with fewer neurons, 256 vs 512 in Model 1, and this may have potentially limited its learning ability. Thus this performed worse than our Model 1 and Model 3.

I was able to get Model 2 to reach 50.27% at one point during training, but when I moved to computing training loss in evaluation mode instead of using the running loss calculated inside the training loop, my accuracy decreased to 46.46%. I think this happened because eval mode disables dropout and batch normalization, so all neurons are available and none are randomly turned off, which can result in more early stopping due to a stricter loss estimate. Also, in the training loop data is taken in batches and not the entire dataset at once so loss being computed is not true loss.